Lycée Guillaume Kroll

INTERNSHIP REPORT
AT ARHS DEVELOPMENTS

CLOUD ENGINEER INTERN

DURAES VALADARES David

Establishment: Lycée Guillaume Kroll

Company: ARHS Developments

Internship tutor: Mr. Dimitri Appriou

Referent teacher: Mr. Cédric Defoy

03/2026 – 06/2026

Acknowledgements

I would like to thank Mr. Dimitri Appriou for supervising me throughout the internship and for the time spent reviewing my work and explaining engineering decisions and cloud practices.

I would also like to thank the members of the Cloud Competence Centre team at ARHS Developments for their support, advice, and welcoming atmosphere during the internship.

Finally, I would like to thank my teachers at Lycée Guillaume Kroll for their guidance during the BTS Cloud Computing course.

Declaration of Honour

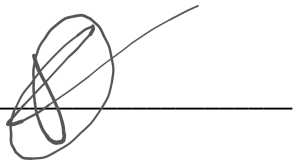
I the undersigned solemnly declare that the internship report submitted for the BTS Cloud Computing at Lycée Guillaume Kroll is based on my own work carried out during my internship at ARHS Developments.

I assert that the statements made and conclusions drawn are an outcome of my own experience and observations. I further certify that:

- The work described in this report was carried out by me during the internship period.
- I have not copied from any source without proper attribution.
- The confidentiality of all professional information has been respected in full.

DURAES VALADARES David

Date: 19/06/2026

Signature: 

Validated by the company tutor:

Name: Dimitri Appriou - ARHS Developments

Date: 19/06/2026

Signature: _____

Abstract

This report describes the 12-week internship carried out at ARHS Developments from 30 March to 19 June 2026, as part of the final semester of the BTS Cloud Computing at Lycée Guillaume Kroll.

The internship took place within the Cloud Competence Centre (CCC), where I contributed to the Project Delivery stream. The main objective was to design, deploy, and automate a highly available Drupal platform on Amazon Web Services (AWS). Drupal is an open-source content management system used to build and manage websites.[1] The objective was to go beyond a single Drupal installation and build the AWS platform needed to deploy, rebuild, monitor, and operate it.

The platform was implemented using infrastructure-as-code and automation tools. Terraform was used to define the AWS infrastructure, while Packer and Ansible were used to build and configure reusable Drupal application images. Deployments were managed through a GitLab CI/CD pipeline with validation, image build, planning, and apply stages. With this pipeline, I could check the Terraform plan before applying changes manually.

The final architecture included an Application Load Balancer, an Auto Scaling Group across two Availability Zones, Amazon EFS for shared Drupal files, and an RDS MySQL database with a read replica in a second private data subnet. I improved security by using restrictive Security Groups, IAM permissions, AWS Secrets Manager, AWS WAF, and WAF log delivery to Amazon S3 through Kinesis Firehose. Monitoring was centralised through a CloudWatch dashboard covering load balancing, compute, database, storage, and WAF metrics.

Around the middle of the internship, the platform reached a first operational stage. The following weeks were used to improve the environment with better security, monitoring, documentation, automation, and a dashboard for daily operations. I also built a separate operations dashboard to monitor the infrastructure, trigger common actions, and test failure scenarios more easily.

During the second half of the internship, I also tested Kiro, an AWS AI-assisted IDE. The objective was to see where Kiro could actually help in the project, especially for documentation, debugging, infrastructure code, and understanding the repository.

The following sections describe where I worked, how the platform was built, the issues I faced, and what I learned from the project.

[1] Drupal - Open-source content management system. Official documentation: https://www.drupal.org/

Table of Abbreviations

ACL	Access Control List
ACM	AWS Certificate Manager
AI	Artificial Intelligence
ALB	Application Load Balancer (AWS)
AMI	Amazon Machine Image
API	Application Programming Interface
ARHS	Arŋs (the company hosting the internship)
ASG	Auto Scaling Group (AWS)
AWS	Amazon Web Services
AZ	Availability Zone (AWS)
BTS	Brevet de Technicien Supérieur
CCC	Cloud Competence Centre (the team at ARHS)
CI/CD	Continuous Integration / Continuous Deployment
CLI	Command-Line Interface
CMS	Content Management System
CPU	Central Processing Unit
DB	Database
EC2	Elastic Compute Cloud (AWS virtual machines)
EFS	Elastic File System (AWS shared storage)
HA	High Availability
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
I/O	Input / Output
IAM	Identity and Access Management (AWS)
IDE	Integrated Development Environment
IP	Internet Protocol (address)
IT	Information Technology
JSON	JavaScript Object Notation
Kiro	AWS AI-assisted IDE used during the second half of the internship
MCP	Model Context Protocol
MFA	Multi-Factor Authentication
MR	Merge Request
NAT	Network Address Translation
OS	Operating System
OWASP	Open Worldwide Application Security Project
PHP	PHP Hypertext Preprocessor
RDS	Relational Database Service (AWS managed databases)
S3	Simple Storage Service (AWS)
SLA	Service Level Agreement
SQL	Structured Query Language
SSH	Secure Shell
SSM	AWS Systems Manager
TLS	Transport Layer Security
URL	Uniform Resource Locator
VPC	Virtual Private Cloud (AWS)
WAF	Web Application Firewall (AWS)
WSL	Windows Subsystem for Linux
XSS	Cross-Site Scripting

Table of Contents

1. Company Presentation

1.1 History and Areas of Activity

ARHS Developments is an IT services and consulting company based in Belval, Luxembourg. The company is part of Arŋs Group, which was founded in Luxembourg in 2003[2] and grew into one of the major technology service providers in the European market. In July 2024, Accenture completed the acquisition of Arŋs Group[3], adding more than 2,330 employees specialised in areas such as software development, business intelligence, data science, artificial intelligence, security management, cloud, and mobile development.

ARHS mainly supports large organisations in the design, development, deployment, and operation of information systems. Its customers include public institutions, European organisations, financial actors, and enterprise clients. The company is especially known for managing complex IT projects where reliability, security, maintainability, and long-term support are important. These projects often involve several technical domains, such as application development, infrastructure, cloud platforms, cybersecurity, data management, and managed services.

The company's activities cover a wide range of IT services. In software development, ARHS designs and builds applications adapted to customer needs. In infrastructure and cloud computing, it helps customers deploy and operate scalable environments on platforms such as Amazon Web Services. In cybersecurity, the company supports the protection of systems, data, and applications through secure architecture, access control, monitoring, and compliance practices. ARHS also provides managed services, meaning that some teams continue to monitor, maintain, and support customer environments after the initial project delivery.

During my internship, I joined the Cloud Competence Centre (CCC) within ARHS Developments. This team focuses on cloud engineering activities, including infrastructure design, automation, deployment, security hardening, monitoring, and operational support. The CCC works both on project delivery and on managed services. Project delivery focuses on building and handing over cloud solutions, while managed services focuses on operating and maintaining existing customer environments.

This matched my project well, because the platform required cloud infrastructure, automation, security, monitoring, documentation, and operational tooling - all areas that match the company's fields of activity.

[2] ARHS Group - Company information and areas of activity. https://www.arhs-group.com/

[3] Accenture - Acquisition and integration information concerning ARHS Group. https://www.accenture.com/

1.2 Organisational Structure

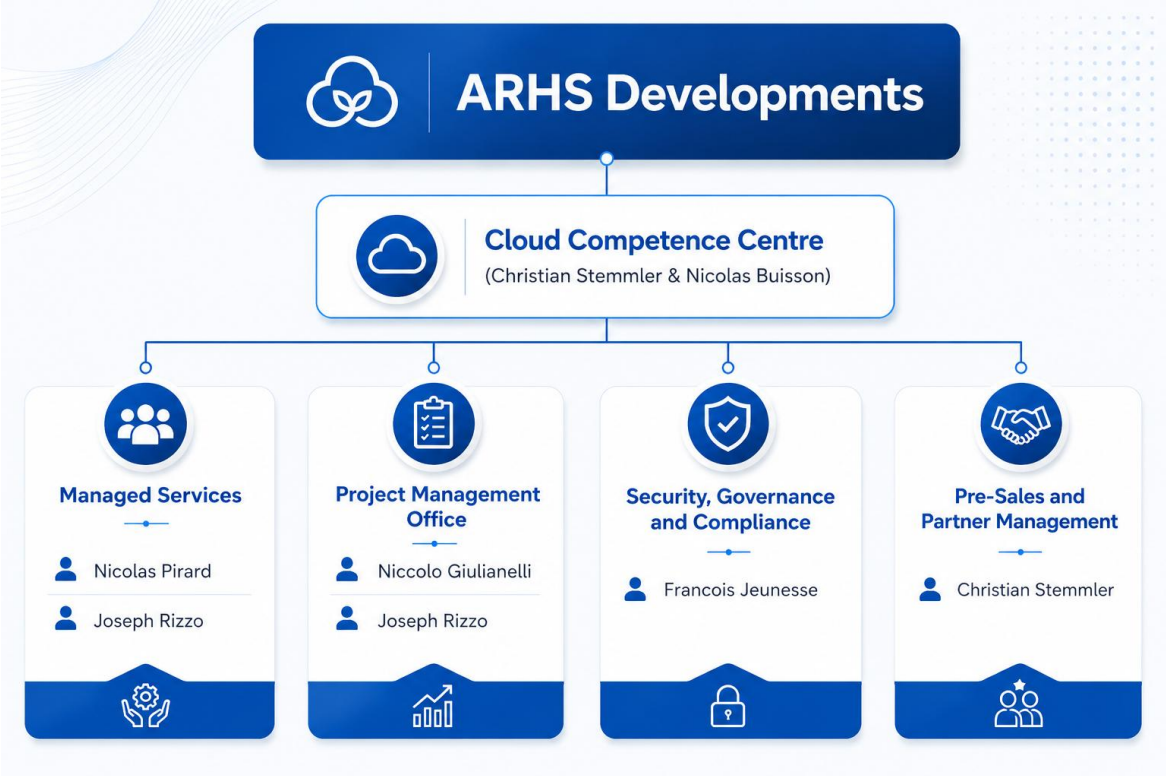


Figure 1 - Organisational structure of the Cloud Competence Centre (ARHS Developments).

2. Integration within the Company

2.1 Description of the Team and Service

The Cloud Competence Centre (CCC) is the team I joined during the internship. It is composed of cloud engineers responsible for designing, deploying, and operating cloud infrastructure for enterprise customers. The CCC is divided into two main streams:

- Project Delivery: responsible for the end-to-end delivery of cloud solutions, from architecture design to deployment and handover to the customer.
- Managed Services: responsible for the monitoring and operation of customer cloud environments, including incident management, maintenance, and SLA reporting.

During the internship, I worked within the Project Delivery stream under the supervision of my company tutor, Mr. Dimitri Appriou.

2.2 Regular Tasks

Throughout the internship, my recurring responsibilities included:

- Following structured online training on Pluralsight covering AWS fundamentals, Terraform, cloud architecture, and Drupal.
- Writing and maintaining Terraform and Ansible code for infrastructure deployment and automation.
- Researching technical topics and presenting findings during meetings with my tutor.
- Creating and updating technical documentation related to the infrastructure and deployment workflows.
- Participating in weekly meetings with my company tutor to review progress and define the objectives for the following week.

Each week, my company tutor defined the expected deliverables, and I was responsible for organising my work autonomously in order to complete them before the next review meeting.

2.3 Integration and Onboarding

The first week (30/03 – 03/04/2026) was mainly dedicated to onboarding. I received my equipment, configured my accounts, met the team, and prepared the development environment using WSL, GitLab, AWS CLI with MFA, a password manager, and a Pluralsight subscription.

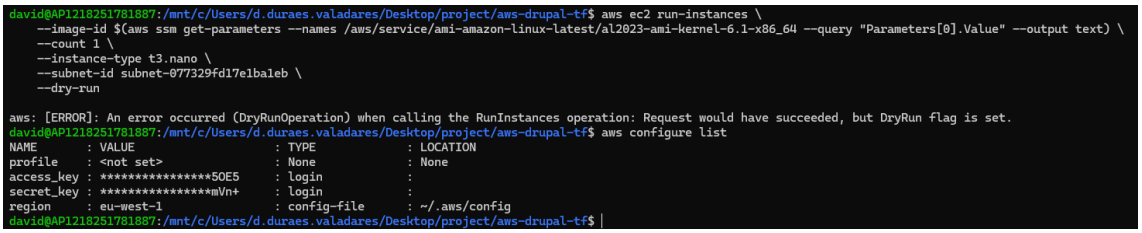


Figure 2 - AWS CLI installed in WSL and a successful dry-run call against the account.

During the first meetings with my tutor, Mr. Appriou, the objectives of the internship were defined. The first tasks focused on completing AWS and Terraform training modules and contributing to the initial project documentation.

One important practice introduced early in the internship was storing the Terraform state inside a shared S3 bucket. This allowed the team and the pipeline to work from the same remote state instead of relying on local state files. Since no DynamoDB locking was configured, concurrent applies still had to be avoided manually. From the second week onward, most of the work focused on the internship project described in Section 3.

3. Internship Project

3.1 Project Description

The objective of the internship project was to build a highly available Drupal platform on AWS with all infrastructure components defined as code. The project included networking, compute resources, storage, security, database infrastructure, monitoring, and deployment automation.

The goal was to make the platform reproducible, so the infrastructure could be deployed again from the GitLab repository with Terraform and the CI/CD pipeline.

During the second part of the internship, Kiro was introduced so I could test how useful an AI-assisted IDE could be on this kind of infrastructure project.

The project repository was organised into five main parts:

- `terraform/` - AWS infrastructure definitions
- `packer/` - image build configuration
- `ansible/` - server configuration and Drupal installation
- `.gitlab-ci.yml` - CI/CD pipeline definition
- `.kiro/` - Kiro configuration and project context for AI-assisted development

The operations dashboard built later in the internship was eventually moved out into a separate companion repository (`aws-drupal-monitor-py`), so that the infrastructure code above and the dashboard could evolve independently. It interacts with this infrastructure only through AWS APIs (boto3) and the GitLab pipeline trigger API, with no shared filesystem.

The next sections explain the architecture and the main technical choices. The annexes contain the detailed technical documents for Terraform, Packer, Ansible, the CI/CD pipeline, GitLab, the dashboard, onboarding, and Kiro. For that reason, I kept the main report focused on the work and moved the deeper implementation details to the annexes.

3.2 Architecture overview

The platform was designed as an automated, two-AZ web infrastructure hosted on AWS.

End-user traffic first passes through AWS WAF before reaching the Application Load Balancer (ALB). The ALB distributes requests across multiple EC2 Drupal instances running inside an Auto Scaling Group spread across two availability zones.

Uploaded files are shared between the instances using Amazon EFS, while application data is stored in an RDS MySQL database deployed in private subnets with a read replica in a second availability zone.

Terraform manages the AWS infrastructure, and GitLab CI/CD handles validation, planning, and deployment steps.

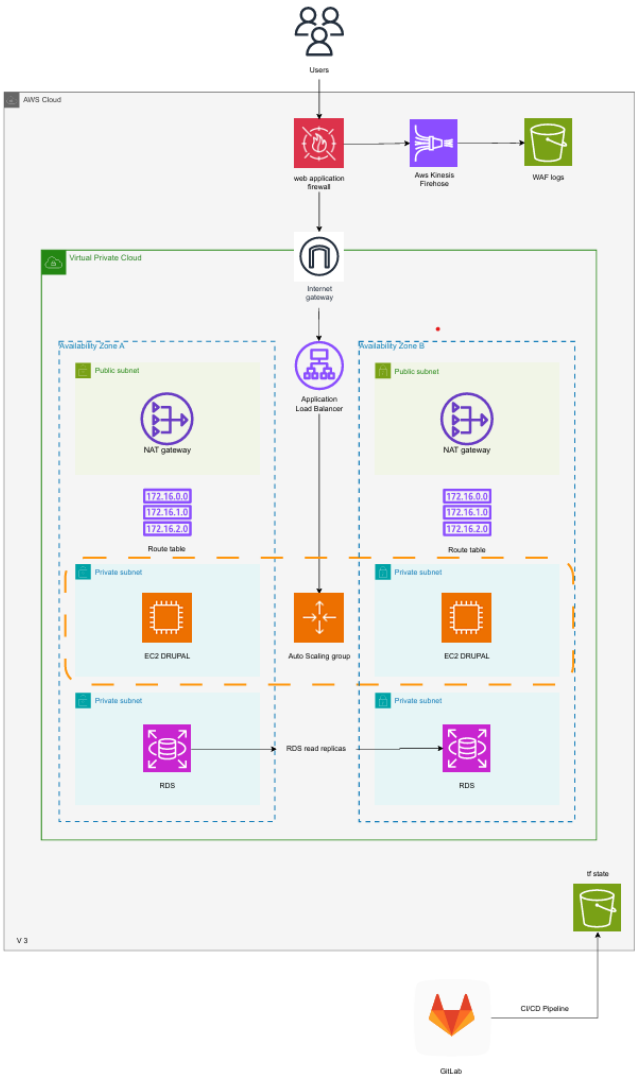


Figure 3 - Infrastructure Architecture Diagram (v3)

3.3 Networking and multi-AZ topology

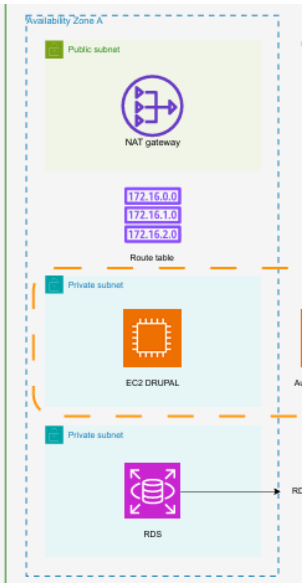


Figure 4 - Architecture excerpt: the public, app, and data subnet tiers replicated in each Availability Zone.

The VPC spans two Availability Zones (AZs) in eu-west-1. An Availability Zone is one or more isolated AWS data centers within a region, designed to reduce the risk of service interruption if one location becomes unavailable. Using multiple AZs is a common high availability (HA) principle in cloud environments: if one AZ fails, components deployed in the second AZ can continue operating, reducing downtime.

The VPC is divided into three subnet pairs: a public subnet, a private app subnet, and a private data subnet in each AZ. The public subnets host the NAT Gateways and the public-facing load balancer nodes; the private app subnets host the EC2 Drupal instances and the EFS mount targets; the private data subnets host RDS. The public subnets route `0.0.0.0/0` through the Internet Gateway. The private application subnets use NAT Gateway routing for outbound access, while the private data subnets do not have a direct internet route. This lets the application servers reach the internet for updates and AWS API calls while keeping EC2, RDS, and EFS away from direct public access.

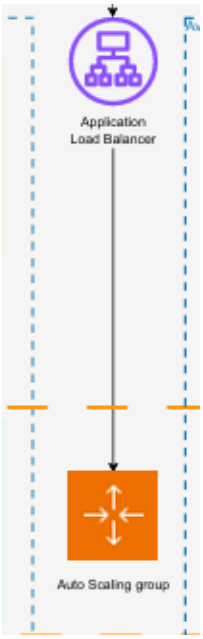
The setup includes an HA toggle exposed as a Terraform variable, `var.HA`. When enabled, a second NAT gateway and Elastic IP are provisioned in AZ B, and the AZ B private route table points to them. With HA enabled, each AZ has its own NAT Gateway for outbound traffic. This improves resilience because resources in the remaining AZ do not depend on a NAT Gateway located in the other AZ. When HA is disabled (the default during development and testing), both private application subnets route through a single NAT Gateway in AZ A, reducing infrastructure costs compared to a production environment.

Figure 5 shows the resulting VPC layout in the AWS console, including the three subnet pairs spread across eu-west-1a and eu-west-1b, and the route tables that connect each subnet either to the Internet Gateway or to its corresponding NAT gateway.

Figure 5 - VPC, subnets and route tables across the two availability zones.

3.4. Compute and load balancing

Figure 6 - Architecture excerpt: the Application Load Balancer distributing traffic to the Auto Scaling group.



The Application Load Balancer is internet-facing and lives in the public subnets. It terminates TLS with a self-signed certificate provisioned by Terraform. This is acceptable for development and testing, but a real ACM certificate would replace it in a public or production environment. The HTTP listener on port 80 redirects traffic to HTTPS with a 301 redirect, while the HTTPS listener on port 443 forwards requests to the target group. To ensure availability, the load balancer performs health checks on the `/health` endpoint of each instance every 15 seconds.

Behind the ALB, an Auto Scaling group of EC2 instances runs Drupal on Amazon Linux 2023. The launch template uses an AMI baked by Packer (see the CI/CD section below) and attaches the same IAM instance profile to every instance, so they can all be reached via SSM Session Manager and pull secrets from AWS Secrets Manager. CloudWatch alarms add capacity when the average CPU exceeds 50 % and remove it when CPU drops below 30 %, with cooldown windows that stop the group from flapping in response to short spikes.

Every instance mounts a shared Amazon EFS file system into Drupal's files directory. This is what makes the file-upload part of Drupal work across several instances: a file uploaded on instance A is immediately visible to instance B, and the load balancer can route requests to either node without losing access to uploaded files.

Figure 7 captures the normal steady state: two instances in the Auto Scaling Group, both reporting healthy in the ALB target group. The CloudWatch alarms and dashboard described in Section 3.8 help surface unhealthy targets, missing instances, or scaling behaviour outside the expected state.

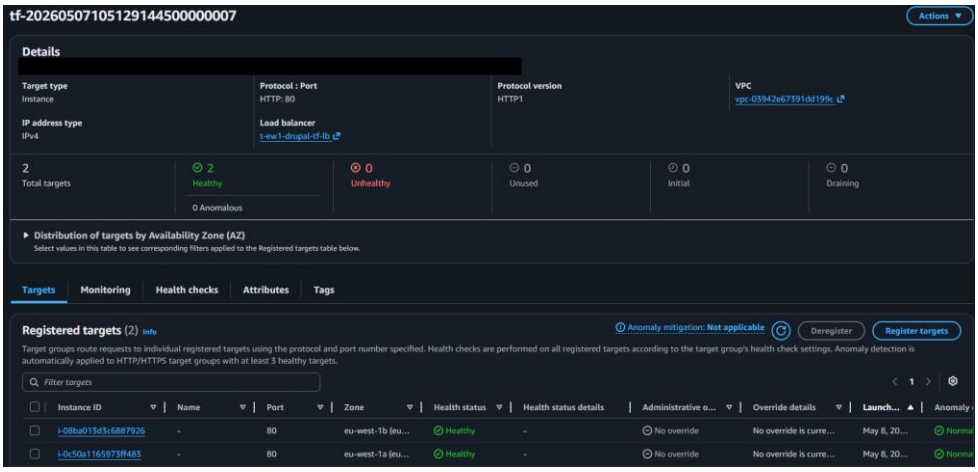


Figure 7 - Auto Scaling group with two in-service Drupal instances and a healthy ALB target group.

3.5 Database and secrets

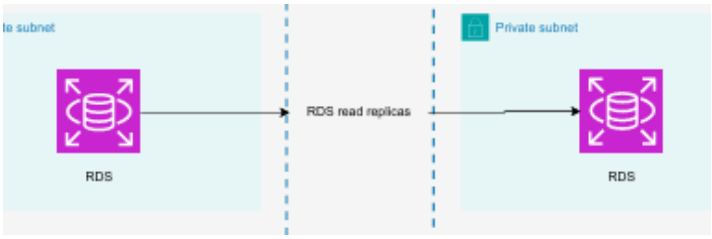


Figure 8 - Architecture excerpt: the RDS primary in AZ A and its read replica in AZ B.

The data tier is an RDS MySQL instance in the AZ A private data subnet, with a read replica in AZ B for read scaling and as a warm-standby option. If the primary database becomes unavailable, the replica could be promoted manually, but this is not the same as automatic Multi-AZ failover. The DB security group only accepts traffic on port 3306, the default port used by MySQL, and only from the EC2 application security group. This ensures that the database is unreachable from the public internet or from other resources in the same VPC.

```
PS C:\Users\d.duraes.valadares\Desktop\aws-drupal-tf> Test-NetConnection -ComputerName t-ew1-drupal-tf-rds-main.cswepsvatygn.eu-west-1.rds.amazonaws.com -Port 3306
WARNING: TCP connect to (10.0.5.161 : 3306) failed
WARNING: Ping to 10.0.5.161 failed with status: DestinationHostUnreachable


ComputerName            : t-ew1-drupal-tf-rds-main.cswepsvatygn.eu-west-1.rds.amazonaws.com
RemoteAddress           : 10.0.5.161
RemotePort              : 3306
InterfaceAlias          : Wi-Fi
SourceAddress           : 10.14.1.77
PingSucceeded           : False
PingReplyDetails (RTT)  : 0 ms
TcpTestSucceeded        : False
```

Figure 9 - TCP connection test to the RDS MySQL endpoint on port 3306 fails from outside the VPC, confirming the database is not publicly exposed.

The master password is generated by Terraform with `random_password` and stored in AWS Secrets Manager.

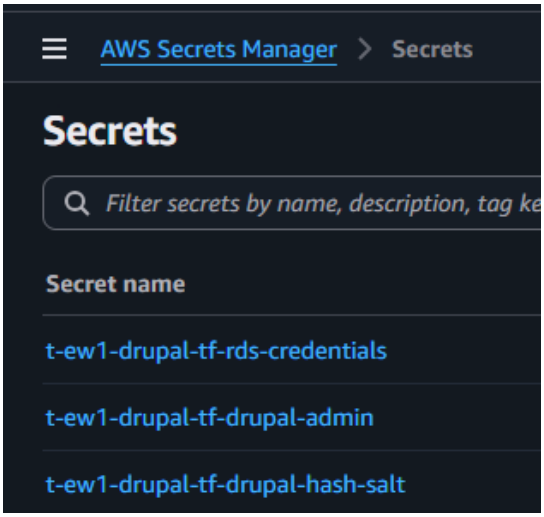


Figure 10 - AWS Secrets Manager: the three project secrets used by the platform.

3.6 Web Application Firewall (WAF)

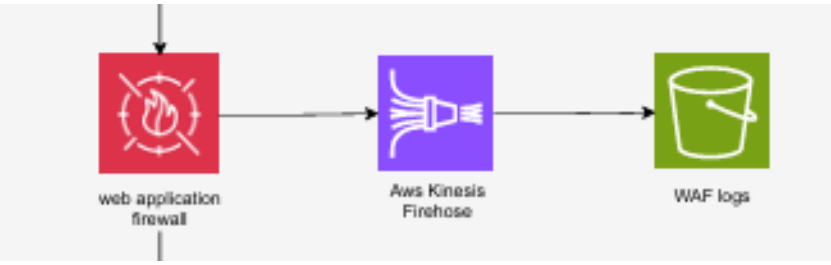


Figure 11 - Architecture excerpt: WAF screens traffic before the ALB; events flow through Kinesis Firehose to S3.

A WAFv2 web ACL sits in front of the ALB and screens every HTTP request before it reaches Drupal. It runs four rules in priority order:

- Priority 1; Amazon IP Reputation List (AWS-managed): blocks traffic from IP ranges known to AWS for scanning, brute-force activity, and other malicious behaviour.
- Priority 2; AWS Managed Rules Core Rule Set: helps block common web attack patterns, such as XSS payloads, suspicious user agents, and oversized request bodies.
- Priority 3; AWS SQL Injection Rule Set: blocks requests containing SQL injection patterns. Drupal already includes protection at application level, but the WAF adds an extra filtering layer before requests reach the ALB.
- Priority 4; Custom Geo Block: blocks all traffic from a configurable list of country codes (currently RU, KP, CN). The country list lives in a Terraform variable, so adjusting it is a one-line change rather than a rule rewrite.

Two body-related rules, `SizeRestrictions_BODY` and `CrossSiteScripting_BODY`, were put in count mode because they could block legitimate Drupal administration actions. This kept visibility in the WAF logs without breaking content editing.

WAF logging sends request records to a Kinesis Firehose delivery stream, which batches them and writes them into a dedicated S3 bucket for later review.

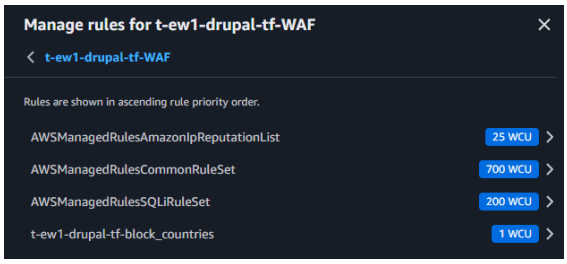


Figure 12 - WAF rule chain attached to the ALB, showing the configured rules and their WCU capacity.

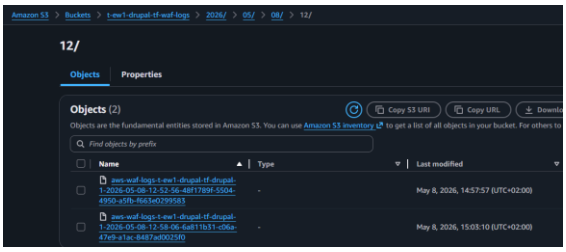


Figure 13 - S3 bucket receiving WAF events delivered by Kinesis Firehose.

3.7 CI/CD pipeline and image bake

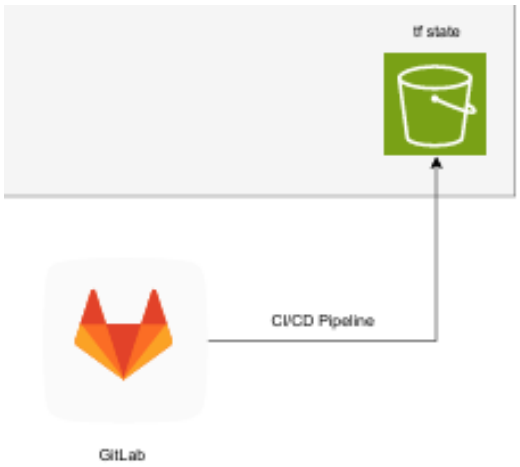


Figure 14 - Architecture excerpt: GitLab runs the CI/CD pipeline, while Terraform state is stored remotely in S3.

The GitLab CI/CD pipeline has four stages: validate, packer, plan, and apply. The validate stage runs `terraform fmt` and `terraform validate` on every push. The Packer stage rebuilds the Drupal AMI only when the commit message contains the word "packer". This is done with a GitLab CI rules condition using the regular expression `$CI_COMMIT_MESSAGE =~ /packer/i`, which makes the check case-insensitive. The plan stage produces a saved Terraform plan as a pipeline artefact; the apply stage consumes that artefact and is gated as manual, so no infrastructure change happens without a human approving it.

The runner authenticates to AWS using minimal bootstrap credentials stored as masked GitLab CI variables. These credentials are used only to call `aws sts assume-role` and obtain a short-lived session token for the job, valid for 3600 seconds. After role assumption, Terraform uses the temporary credentials for the rest of the job.

The Drupal application image itself is created by Packer, driven by an Ansible playbook that installs PHP, nginx, Drupal core, the EFS mount helper, and a few small operator helper scripts: `db-connect` for opening a MySQL shell from inside an EC2 instance over SSM, and `drupal-pw` for printing the auto-generated admin credentials. The output is an AMI tagged with a known name; Terraform's `aws_ami` data source looks up the most recent AMI with that tag and feeds it into the EC2 launch template, so after `terraform apply` updates the launch template, new or refreshed instances use the latest selected image.

Figure 15 captures one of these pipeline runs. The pipeline stages are visible in order, and the apply stage is configured as a manual gate that an operator runs with the ▶ play button.

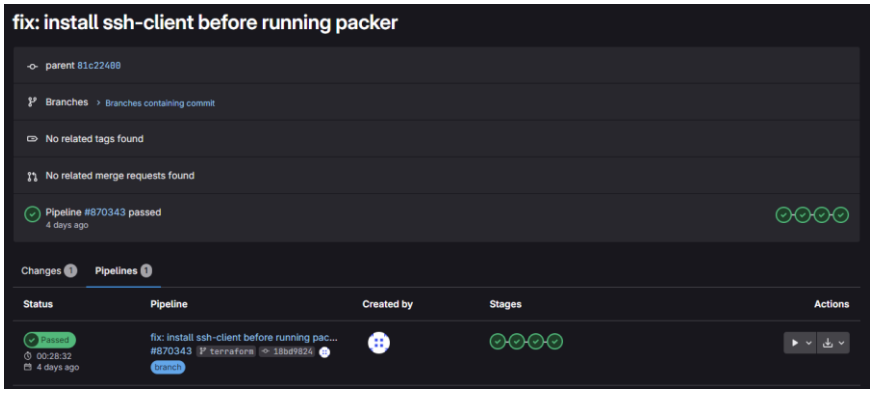


Figure 15 - GitLab pipeline run with the manual-gated apply stage.

Figure 16 shows the same pipeline as a diagram, mapping each stage to its main output or action so the relationship between the CI/CD steps and the deployed infrastructure is visible at a glance.

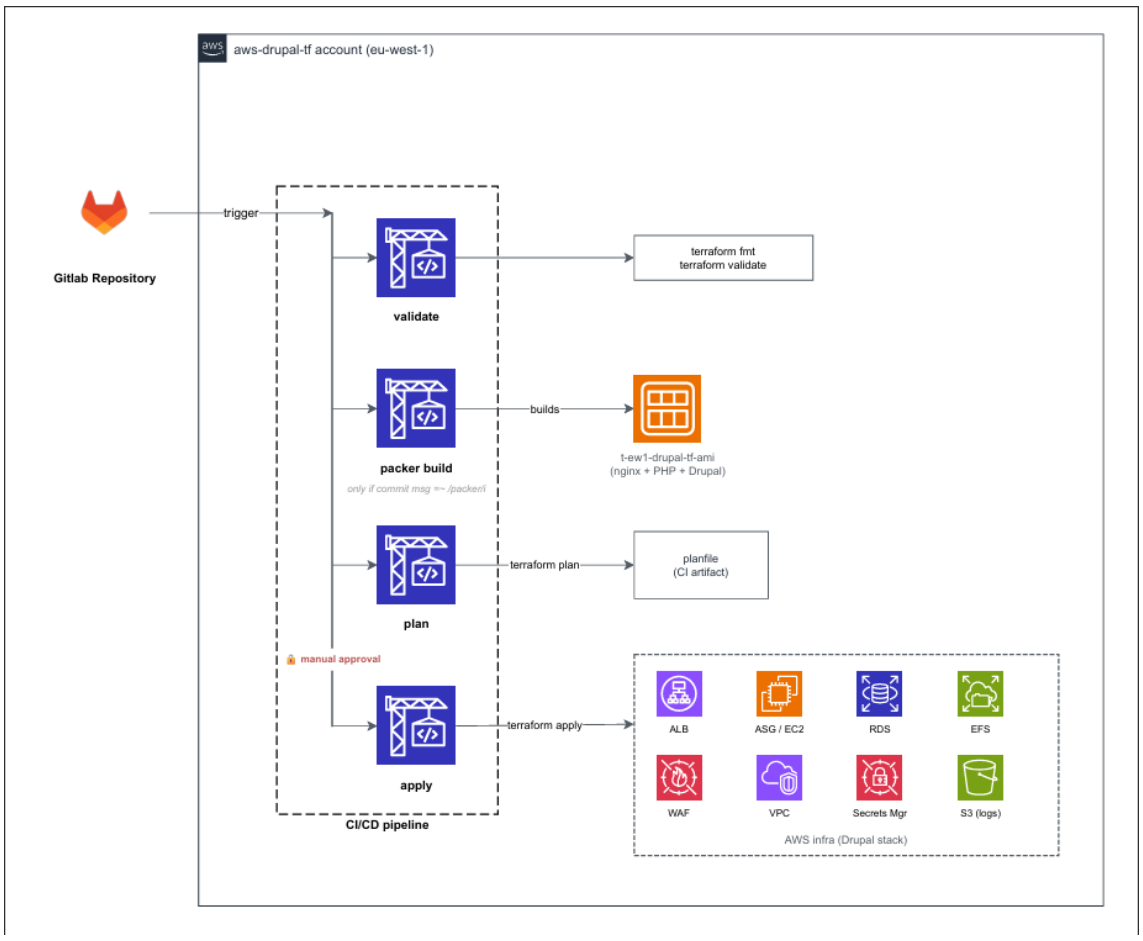


Figure 16 - Pipeline architecture diagram: validate → packer → plan → apply, mapped to the main output of each stage.

3.8 Observability - CloudWatch dashboard

The main operational metrics from the platform are gathered on a CloudWatch dashboard that Terraform creates and keeps in sync with the rest of the infrastructure. The dashboard has six sections: ALB traffic and latency, ALB target health and connections, ASG / EC2, RDS, EFS, and WAF. The ASG / EC2 section includes CPU with the 50 % scale-out and 30 % scale-in thresholds, instance count, and network I/O.

Figure 17 zooms into the first row of that dashboard, the ALB Traffic and Latency panels, captured under a synthetic load so the request-count and latency series have something to draw rather than the flat lines you would otherwise get from a sleeping environment.

Figure 17 - CloudWatch dashboard: ALB traffic and latency panels under load.

4. Project Approach and Technical Challenges

I built the platform one piece per week, with a merge request at the end of each iteration that Mr. Appriou reviewed before I moved on. That cadence, design on Monday, code Tuesday to Thursday, merge on Friday, review with Mr. Appriou the following Tuesday, ended up being the rhythm of the whole internship. The sections below walk through the project chronologically.

4.1 Infrastructure foundations

The first technical step was to put the base AWS infrastructure in place. After reviewing the architecture with my tutor, I started with the parts that everything else depended on: the VPC, the public and private subnets, the Application Load Balancer, the Auto Scaling Group, the security groups, and SSM access. I also prepared Terraform in WSL and created the shared S3 bucket used to store the Terraform state. From that point, the main infrastructure could be changed through code instead of being created manually in the AWS console.

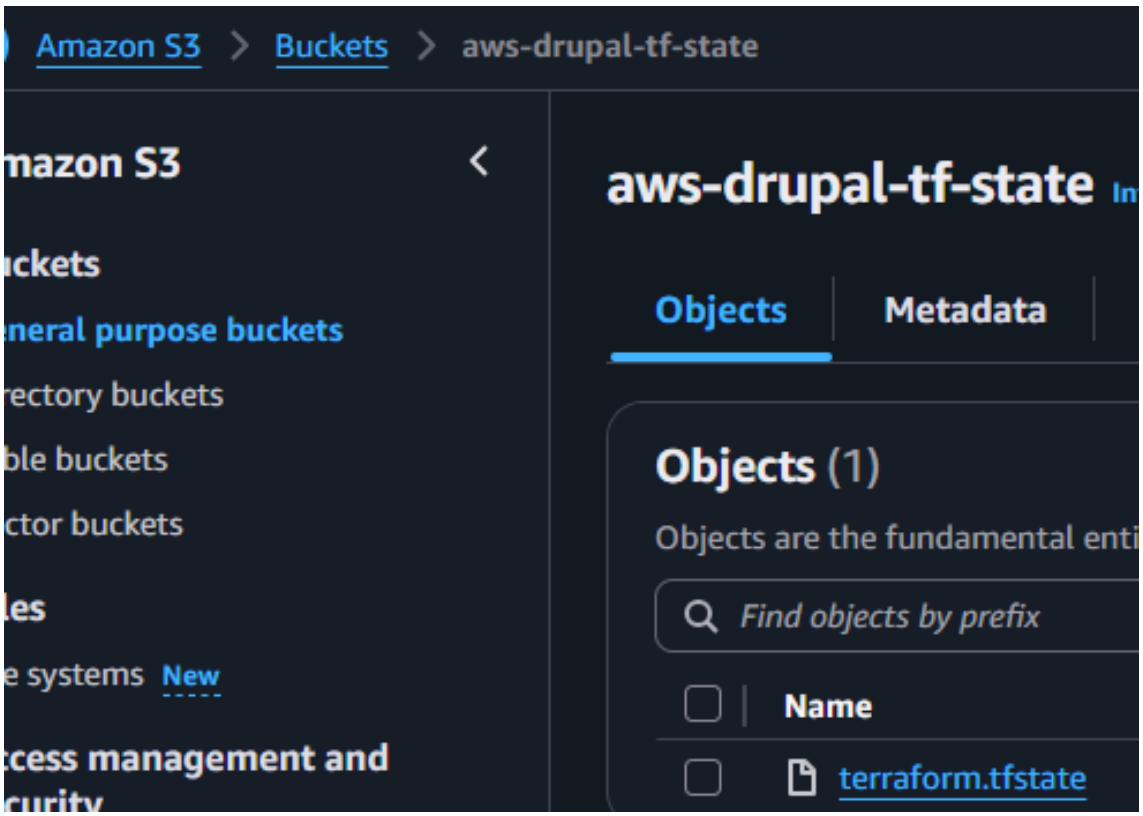


Figure 18 - Manually created S3 bucket holding the shared Terraform state.

The first significant issue appeared when the ALB target group stayed unhealthy. The EC2 `user_data` script was supposed to install nginx so that the health check had a local web service to reach, but the installation was not completing correctly. After checking the instance logs and the network rules, I found

that the EC2 security group only allowed outbound traffic on port 443. The package installation also needed port 80 to reach the repositories. After allowing the required outbound traffic, the health checks became healthy.

4.2 Security, database, and deployment pipeline

After the base infrastructure was working, the next step was to improve security and prepare the database layer. Following a discussion with Mr. Appriou, I changed the EC2 image from Ubuntu to Amazon Linux 2023 to stay closer to the AWS ecosystem. I also restricted inbound access to the two ARHS office IP ranges and added AWS WAF in front of the Application Load Balancer. The WAF used three managed rule sets: Amazon IP reputation, AWS Core Rule Set, and SQL injection protection. I tested the geo-blocking rule by temporarily blocking traffic from Russia and checking the result in the WAF console. The rule was later extended to North Korea and China.

To keep a record of allowed, counted, and blocked requests, I configured WAF log delivery to a dedicated S3 bucket through Kinesis Firehose. I then added the database tier with an RDS instance in a private subnet and a dedicated security group. In the same phase, I created the first GitLab CI/CD pipeline for the project. It ran `terraform fmt`, `terraform validate`, `terraform plan`, and `terraform apply`, which made the deployment process repeatable from the repository instead of being executed manually.

4.3 Application deployment and secret management

After the infrastructure, security, and database layers were in place, the next step was to prepare the platform for Drupal and improve how secrets were handled. The database password was generated with Terraform using `random_password` and stored in AWS Secrets Manager instead of being kept in a `.tfvars` file, script, or Git commit. For the RDS password, I disabled special characters to avoid connection-string issues; the Drupal admin password remained a separate generated secret. The access pattern was documented in the README, including a helper script for legitimate debugging.

I also changed how the GitLab pipeline authenticated to AWS. The first approach relied too much on long-lived access keys, which was not ideal. I tested IAM role assumption, but it did not work correctly with the existing GitLab runner setup. The retained solution was to keep an access key in GitLab CI variables only for the initial authentication step, then use `aws sts assume-role` to obtain a short-lived session token. Terraform then used the temporary credentials for the rest of the pipeline.

In parallel, I started preparing the Drupal application layer. I first followed Drupal training material, then installed Drupal manually on an EC2 instance to understand its behaviour before automating the installation with Packer and Ansible.

4.4 High availability for Drupal

Once Drupal worked on one EC2 instance, the next challenge was making it work correctly with several instances behind the load balancer. A single-instance setup was not enough for the project objective, because the Auto Scaling Group had to be able to replace or add instances without breaking the website.

The first issue was shared storage. Drupal stores uploaded files locally by default, which caused a problem in a multi-instance setup: content uploaded through one EC2 instance did not automatically appear on the other one. The retained solution was to mount Amazon EFS on each Drupal instance and use it for Drupal's shared files directory.

The second issue was consistency between instances. Sessions were invalidated when the load balancer switched from one instance to another. To solve this, I used the same Drupal `hash_salt` value across the cluster and connected all instances to the same RDS database.

I also compared three deployment approaches with Mr. Appriou: a large `user_data` bootstrap script, configuration on boot with Ansible, and a pre-baked AMI built with Packer. The pre-baked AMI approach was retained because it made new instances faster and more predictable. By the end of this phase, the same Drupal content was visible from both EC2 instances regardless of which one the load balancer selected.

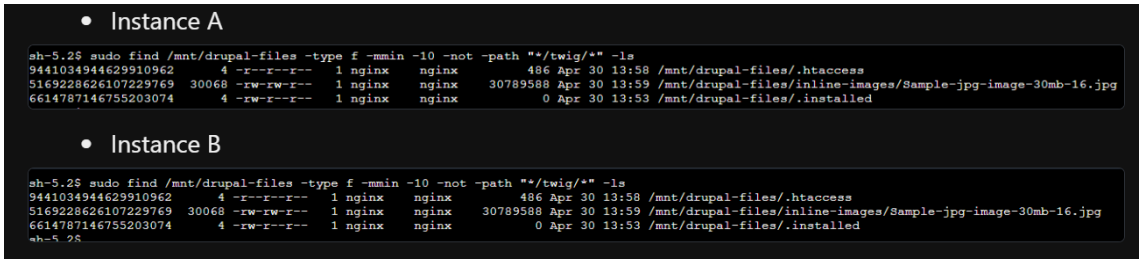


Figure 19 - The same Drupal content served from instance A and instance B, confirming shared storage across the instances.

4.5 Image automation and first Kiro tests

This phase focused on cleaning up the Packer and Ansible image-build process. I improved the variable definitions, added missing descriptions, and created a cleanup script for failed Packer builds. This was useful because failed manual Packer tests could leave temporary resources behind, including EC2 instances, AMIs, or networking resources. I also added GitLab pipeline rules so the Packer build only ran

when the commit message explicitly mentioned Packer. This avoided rebuilding the Drupal image when only the infrastructure code had changed.

During manual Packer tests, some failed builds left temporary networking resources behind, including orphan VPCs. This pushed the AWS region close to its VPC quota and showed why the build cleanup had to be reliable. I then improved the build script so the temporary VPC, subnet, route table, Internet Gateway, and related resources were removed automatically after the build. No Elastic IP was created for this build network; the temporary subnet used public IP assignment instead.

Two technical issues had to be fixed during this phase. The first one concerned the database password generated by Terraform. Some special characters caused problems in the Drupal connection string, so I disabled special characters in the `random_password` resource. The second issue came from the dependency between Packer and Terraform. Packer needed a VPC and subnet to launch its temporary build instance, but `terraform destroy` removed those resources.

The retained solution was to create a small bash script that provisions a temporary VPC and subnet before the Packer build, exports their IDs for Packer to use, and deletes them after the AMI is created. This kept the image build independent from the main infrastructure lifecycle, so destroying the environment no longer broke the next Packer run.

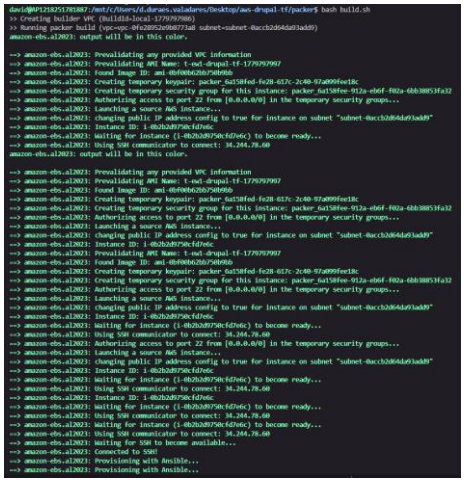


Figure 20 - Bash bootstrap script provisioning a throwaway VPC for the Packer build.

Around this point, the mid-internship review confirmed that the core infrastructure and deployment workflow were operational. The focus then shifted toward monitoring, security hardening, documentation, maintainability, and operational tooling. During the same period, I received access to Kiro and started evaluating how it could support cloud engineering work, especially for documentation, debugging, project understanding, and collaboration.

I configured Kiro with project context and tested MCP integrations for Terraform, AWS documentation, pricing, CloudWatch, and cost analysis. In the final setup, Terraform and AWS documentation stayed enabled, while pricing and CloudWatch were disabled because they were not needed for the final workflow. One important lesson was that AI-assisted changes still had to be reviewed carefully. During a `user_data` refactor, a generated change caused infrastructure issues and had to be rolled back. This helped me better understand the limits of AI assistance in an infrastructure project.

The detailed Kiro configuration and usage are documented in a separate technical report attached in the annexes.

4.6 Observability and operational dashboard

Another improvement was enabling HTTPS on the platform. With the help of Kiro, I configured a self-signed certificate in front of the ALB. While this was not a production-ready solution, it allowed me to validate end-to-end encryption without needing to purchase a domain name.

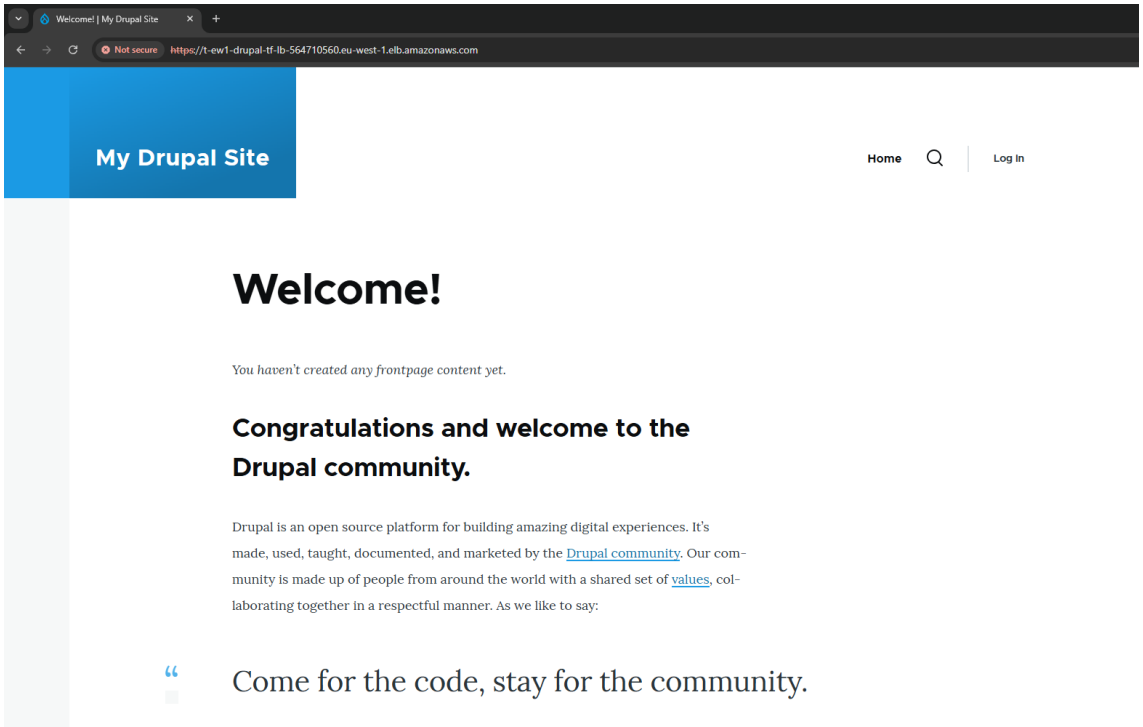


Figure 21 - Drupal site reached over HTTPS through the ALB with the self-signed certificate.

I also started writing this internship report and compared it with the Vademecum BTS-CC-Stage guidelines to make sure the structure matched expectations. Instead of keeping a day-by-day log, I focused more on the challenges faced and the outcomes achieved during the internship. During the Tuesday meeting, Mr. Appriou reviewed an early version and encouraged me to continue improving both

Kiro and the infrastructure hardening. He also asked me to send the full documentation by the end of the week for a more detailed review.

I then worked on monitoring and failure testing to check how the infrastructure reacted when something went wrong. I created a small chaos-testing Bash script with a menu to trigger controlled failure scenarios and see how the system behaved. For example, I could terminate EC2 instances to check if the Auto Scaling Group replaced them correctly, simulate high CPU or memory usage, and test what happened if some services became unavailable. The goal was to see how the infrastructure reacted during failures and check whether the monitoring dashboard showed the problem clearly. It also helped me check if the environment recovered correctly after an issue.

Figure 22 - Interactive chaos-testing menu used to trigger controlled failure scenarios against the live infrastructure.

Because EC2 instances do not expose memory and disk metrics by default, I added a CloudWatch agent configured through a small JSON file. The agent publishes memory and disk usage every 60 seconds and tags the metrics with the instance ID and Auto Scaling Group name. To automate deployment, I updated the Ansible playbook to install the agent automatically, while Terraform was updated to attach the required IAM policy and enable detailed monitoring on the launch template.

4.7 Operational tooling and GitLab integration

During this phase, I encountered a Terraform timeout while applying the infrastructure. The ALB took longer than expected to provision, and Terraform returned a context deadline exceeded error. After checking AWS, I confirmed that the ALB was still being created and that the infrastructure was not actually broken.

To avoid the same issue in future deployments, I added `timeouts { create = "20m" }` to the `aws_lb` resource. After this change, the apply completed successfully and the merge request was approved and merged. While working in the same area, I also changed the EC2 instances from `t3.micro` to `t2.micro` to reduce costs.

The main task in this phase was building an operations dashboard with help from Kiro. The goal was to avoid switching between AWS, GitLab, and terminal commands for common operations. The dashboard displays the status of services such as the ALB, Auto Scaling Group, RDS, EFS, NAT and WAF. It also lists EC2 instances with their state and availability zone, shows CloudWatch metrics (CPU, latency, requests, 5xx errors, RDS and EFS activity), and provides a cost overview for the current month.

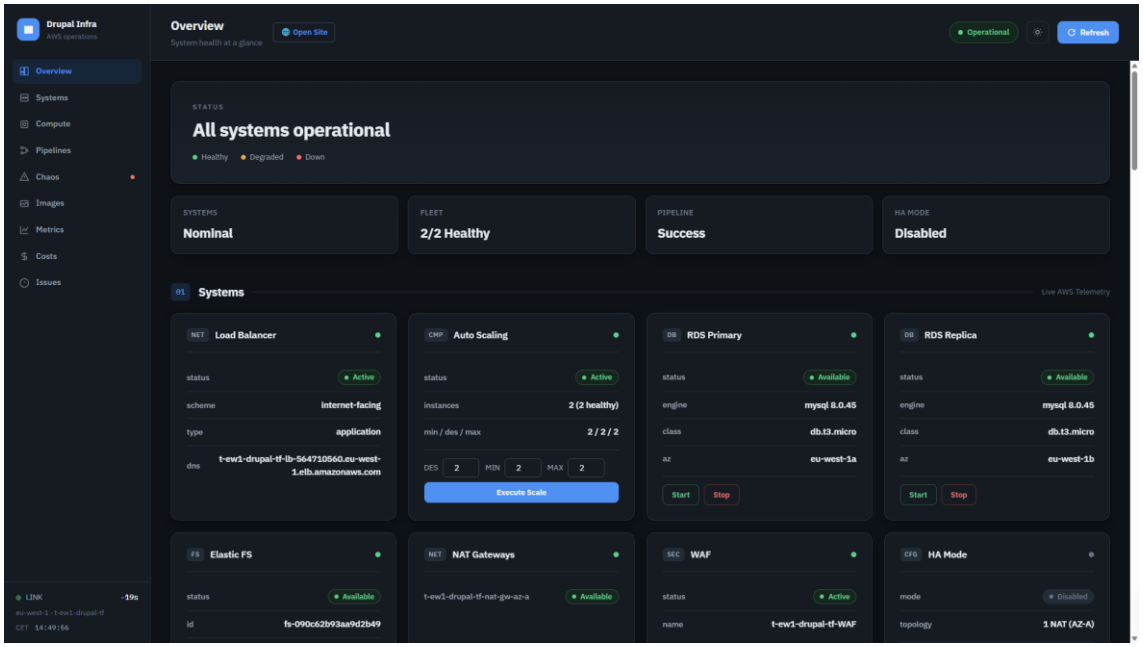


Figure 23 - Operations dashboard overview: service status, EC2 inventory, metrics, and month-to-date cost.

I also added management actions directly in the dashboard. For example, it can scale the ASG, reboot or terminate EC2 instances, stop or start RDS, recreate the read replica, and switch HA mode on or off. A Chaos Engineering section was also included for failure-injection tests, such as instance termination, CPU or memory stress, blocking access to EFS and RDS, and simulating an AZ-A application-layer failure by terminating the EC2 instances running in eu-west-1a. CI/CD features were added as well, including pipeline monitoring, Packer build triggers, and manual job controls. To limit access, the dashboard is password protected and intended for local access only through my development environment.

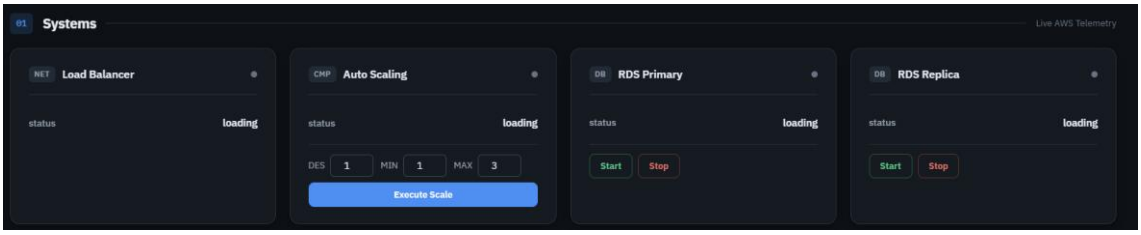


Figure 24 - Controls panel: scale the ASG, reboot or terminate instances, manage RDS, and toggle HA mode.

Figure 25 - Chaos Engineering panel exposing the same failure-injection scenarios as the bash script.

I had several dashboard issues to solve. The most time-consuming one was getting Packer builds to launch correctly from the dashboard through GitLab. The dashboard was designed to trigger the GitLab pipeline through the Pipeline Trigger API, so the build would run inside CI/CD instead of on the operator's machine. The first issue was that the required trigger token was not configured correctly. I also had to make sure the dashboard passed the right trigger variable so the Packer stage would run.

The operations dashboard does not run Packer or Terraform locally. Packer builds and Terraform actions are triggered through the GitLab API and executed inside GitLab CI/CD. The dashboard acts as a control interface: it triggers pipelines, monitors pipeline and job status, can play manual jobs, and can start an ASG instance refresh through boto3 after an AMI deployment action. This keeps AMI builds in the same CI/CD workflow as the rest of the project.

In the final version, the dashboard trigger still relies on passing a Packer rebuild message to the pipeline. A future improvement would be to replace this with an explicit CI variable such as `RUN_PACKER=true`, because that would be clearer and more reliable than depending on the commit-message rule.

I also spent time improving the GitLab integration. At first, the `.env` file was being loaded from the wrong location, the GitLab URL had a trailing slash causing malformed API calls, and the token I was using turned out to be a Feed token (`glft-`) instead of an API token (`glpat-`). What initially looked like a GitLab configuration issue was actually a 401 error caused by the dashboard session/authentication state, not by GitLab itself.

After fixing those problems, I redesigned the pipeline section to make it easier to use. Pipelines were displayed as cards with status indicators and manual action buttons. I also connected the branch selector to the GitLab API and added an option to include a Packer build in a triggered pipeline.

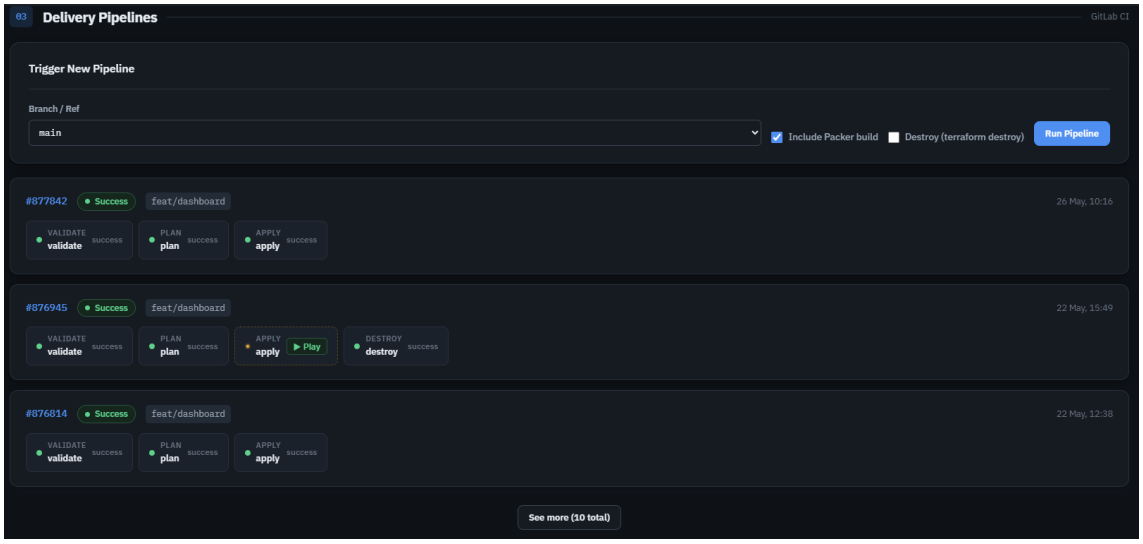


Figure 26 - GitLab pipelines redesigned as cards with inline job chips, status dots, and play buttons for manual jobs.

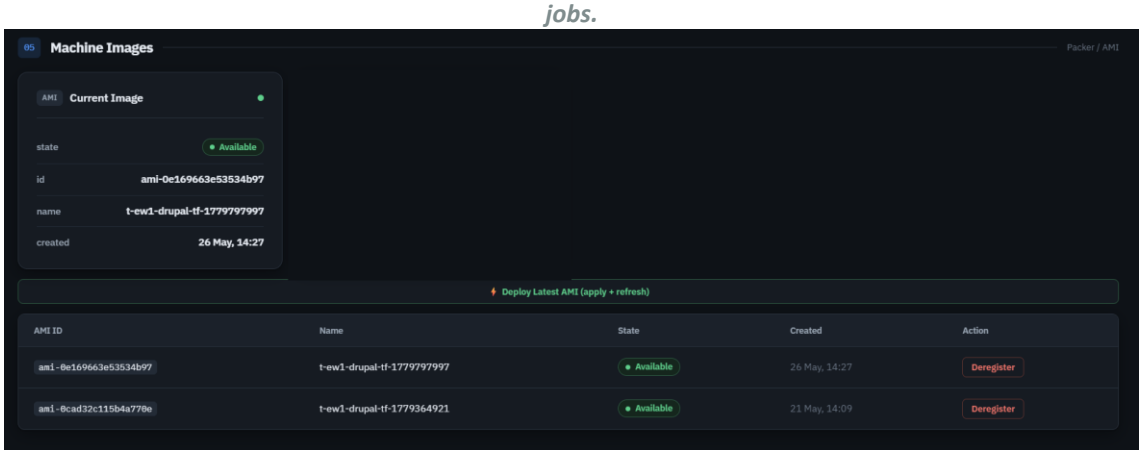


Figure 27 - AMI management view: list of project AMIs with the currently used image highlighted.

The dashboard merge request also showed me why clean Git history matters. My branch had fallen behind main because the previous MR had been squash-merged, which caused Git to no longer recognise some commits as already integrated. My first rebase attempt quickly became messy with several conflicts. I tried merging instead, but the MR became too large and mixed old and new work together, which made it difficult to review.

In the end, I created a fresh branch from main, manually moved only the dashboard-related changes, and pushed a much cleaner commit. From this experience, I learned that it is important to rebase regularly and keep merge requests focused on one feature instead of letting too many changes accumulate.

This phase ended with the pipeline diagram being finalised and the last Terraform lab from the Drupal training being completed.

4.8 Documentation and final preparation

The final phase focused mainly on documentation and preparation for the defence. After the main technical parts were finished, I updated the internship report with the latest progress and added screenshots of the platform.

I also created separate technical documents for the main parts of the project: the project overview, Terraform, Packer, Ansible, the GitLab CI/CD pipeline, the repository split, the operations dashboard, onboarding, and Kiro. These documents go into more detail than the main report. The idea was to keep the report focused on what I did during the internship, the problems I faced, and the solutions I kept, while the annexes explain the implementation more deeply.

The remaining work was to improve the report, prepare the presentation slides, check the previous merge requests, and make sure the documentation matched the final state of the project.

4.9 Results and retained solutions

By the end of the internship, the project resulted in a working and automated Drupal platform on AWS. The infrastructure was managed with Terraform, deployed through GitLab CI/CD, and built around a two-AZ, high-availability-oriented architecture using an Application Load Balancer, an Auto Scaling Group, Amazon EFS, and RDS.

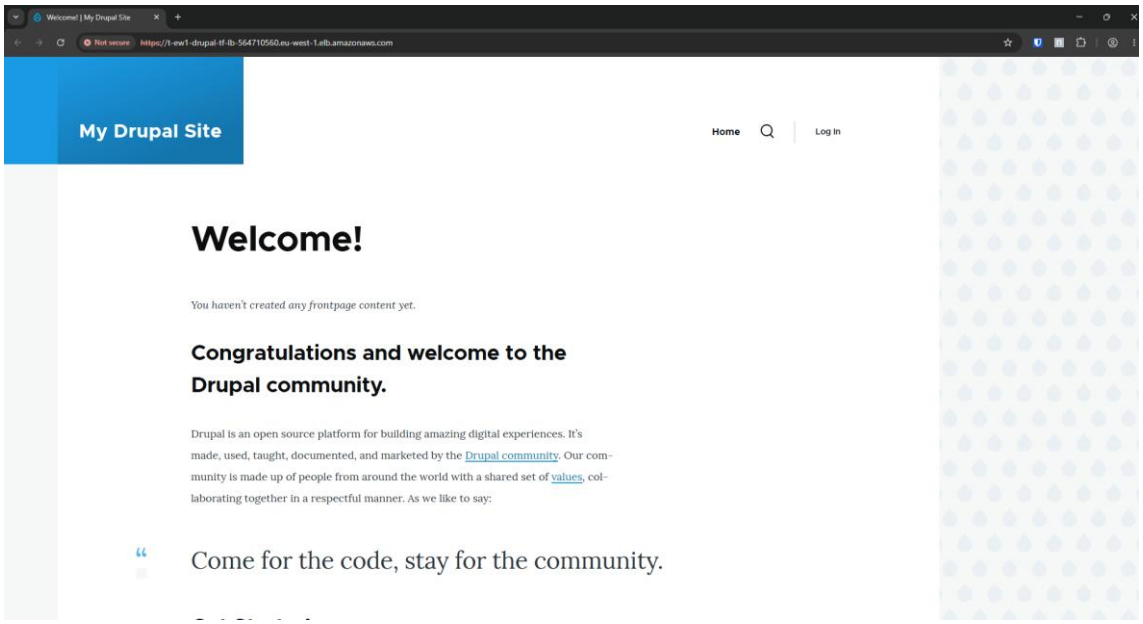


Figure 28 - Drupal site running on the two-AZ platform, served through the ALB.

The main result was not just that Drupal was running. The platform could also be redeployed, updated, monitored, and tested through a defined workflow. The CI/CD pipeline handled validation, planning, image builds, and manual apply steps for infrastructure changes. Security was improved through restricted Security Groups, AWS Secrets Manager, IAM permissions, and AWS WAF with logging to S3.

During the second half of the internship, I added more operational features around the platform. This included a CloudWatch dashboard, chaos-testing scripts, improved documentation, and a separate Python operations dashboard using boto3. These additions made it easier to check the infrastructure state, test failure scenarios, and run common actions without jumping between too many tools.

By the end, the project was more than a deployment exercise. It included infrastructure as code, image automation, security, monitoring, documentation, and operational support.

5. Personal Conclusion

This internship gave me my first real experience working on a complete cloud infrastructure project inside a company. I could finally apply concepts from the BTS in one project: Terraform, AWS networking, load balancing, automation, security, and monitoring.

Technically, I improved the most in understanding how AWS services fit together in a complete environment. At the beginning, I already had some basic knowledge of AWS and Terraform, but this project helped me understand how these tools are used together in a more complete environment. I learned how to design a VPC, deploy resources across multiple Availability Zones, configure an Application Load Balancer, automate infrastructure with Terraform, build reusable images with Packer and Ansible, and secure the platform with services such as IAM, Secrets Manager, Security Groups, and AWS WAF.

I also understood that deploying the application is only the first part. A cloud platform also needs to be secure, observable, maintainable, and easy to operate. During the second half of the internship, I worked more on these aspects by improving the monitoring, running failure tests, writing documentation, and building an operations dashboard. It showed me the difference between a working demo and a platform that another engineer can monitor, troubleshoot, and reuse.

I also learned from the problems I faced. Some issues were technical, such as unhealthy load balancer targets, Terraform timeouts, broken Packer builds, GitLab authentication problems, and dashboard integration issues. These problems forced me to troubleshoot step by step, read documentation carefully, test small changes, and understand the cause before applying a fix. That made me more confident when working independently.

On the professional side, the internship helped me improve my organisation and communication. Each week, I had objectives to complete before the next review meeting with my company tutor. I had to prepare my work, explain my choices, accept feedback, and improve my merge requests and documentation. I also learned the importance of keeping work clean and easy to review, especially when working with Git and CI/CD pipelines.

This internship confirmed my interest in cloud engineering. I enjoyed working on infrastructure, automation, security, monitoring, and operational tooling. If I had more time, I would improve the project further by using a production-ready TLS certificate, adding more automated tests, strengthening the dashboard authentication, and making the deployment workflow closer to a production environment.

By the end of the internship, I was more confident with AWS, Terraform, GitLab, troubleshooting, and explaining my work during reviews. It gave me a clearer view of cloud engineering work in a company and confirmed that I want to keep improving in this field.

(Word count from Company Presentation to Personal Conclusion: 6,042 words)

Bibliography

Official documentation

- AWS Documentation - https://docs.aws.amazon.com/
- Terraform Documentation - https://developer.hashicorp.com/terraform
- Packer Documentation - https://developer.hashicorp.com/packer
- Ansible Documentation - https://docs.ansible.com/
- Drupal Documentation - https://www.drupal.org/docs
- GitLab CI/CD Documentation - https://docs.gitlab.com/ee/ci/

Company sources

- ARHS Group - Company information and areas of activity. https://www.arhs-group.com/
- Accenture - Acquisition / integration information concerning ARHS Group. https://www.accenture.com/

Training and learning resources

- Pluralsight - AWS fundamentals, Terraform deep dives, cloud architecture paths. https://www.pluralsight.com/

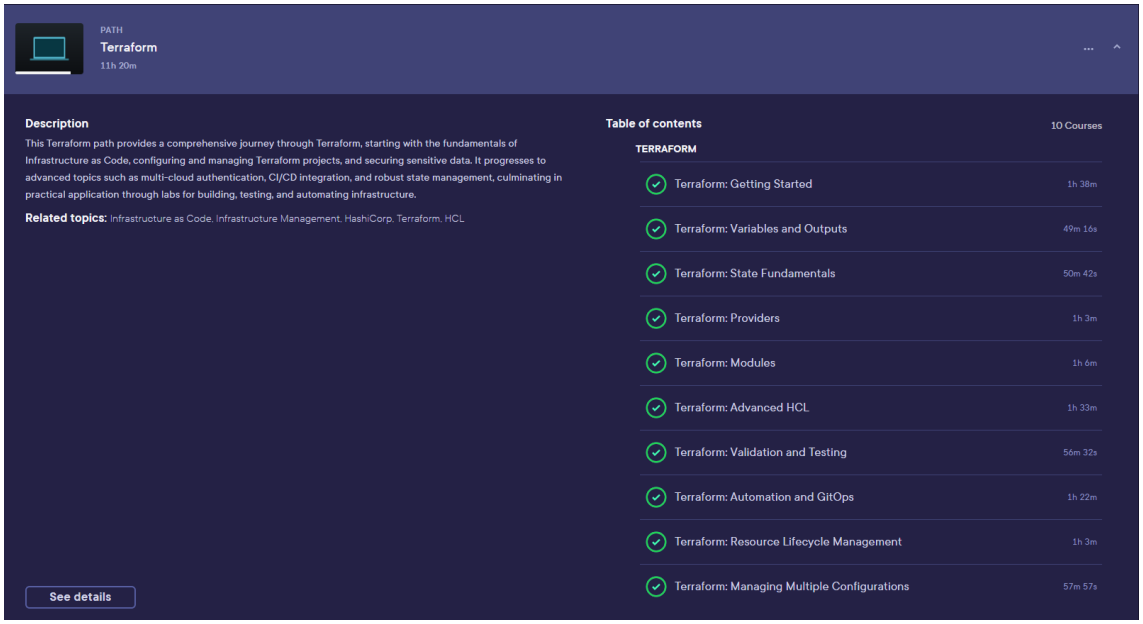


Figure 29 - Pluralsight course completion certificate.

Table of Figures

Annexes

The following annexes are provided as supporting documents. The main report explains the internship context, the project work, the main challenges, and the final result. The annexes give more technical detail for readers who want to understand how each part of the platform was implemented.

The daily activity log is provided as a separate companion file. The other annexes explain the main technical parts of the project in more detail.

- Annex A - Daily Activity Log: companion document recording the day-to-day tasks carried out during the internship (provided as a separate file).
- Annex B - Project Overview: how the whole project fits together, from infrastructure to deployment.
- Annex C - Terraform: how Terraform defines and manages the AWS infrastructure.
- Annex D - Packer: how the Drupal application image is built with Packer.
- Annex E - Ansible: how Ansible configures the servers and installs Drupal.
- Annex F - CI/CD Pipeline: how the GitLab pipeline validates, builds, and deploys the infrastructure.
- Annex G - Repository Split and GitLab Infrastructure: how the project repositories are organised on GitLab.
- Annex H - Operations Dashboard: how the separate operations dashboard works.
- Annex I - Onboarding Guide: the setup steps for a new team member to clone the repositories, authenticate, run the dashboard, and understand the workflow.
- Annex J - Kiro: technical report on the Kiro configuration and usage during the internship.